

Transitions: DS + Algorithms

Tree-based
data structures

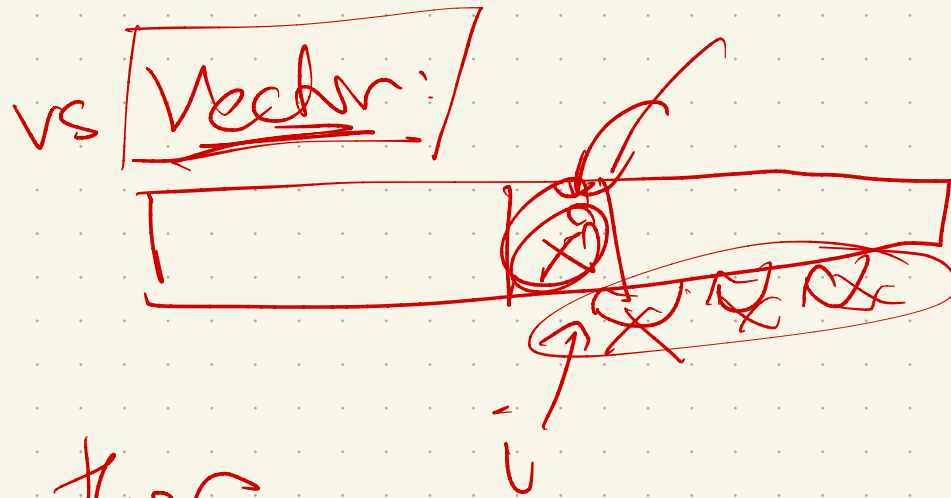


Trade-offs: List vs. Array/Vector

A) Vector: $O(1)$ access
+ push at back

B) Linked: removes/inserts :
 $O(1)$ time

$O(1)$
vs
 $O(n)$

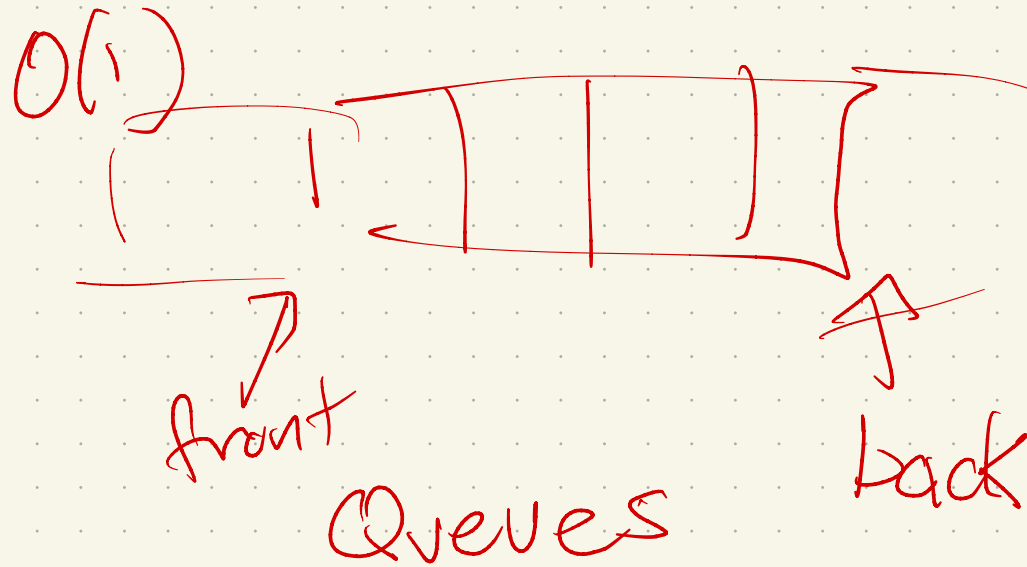
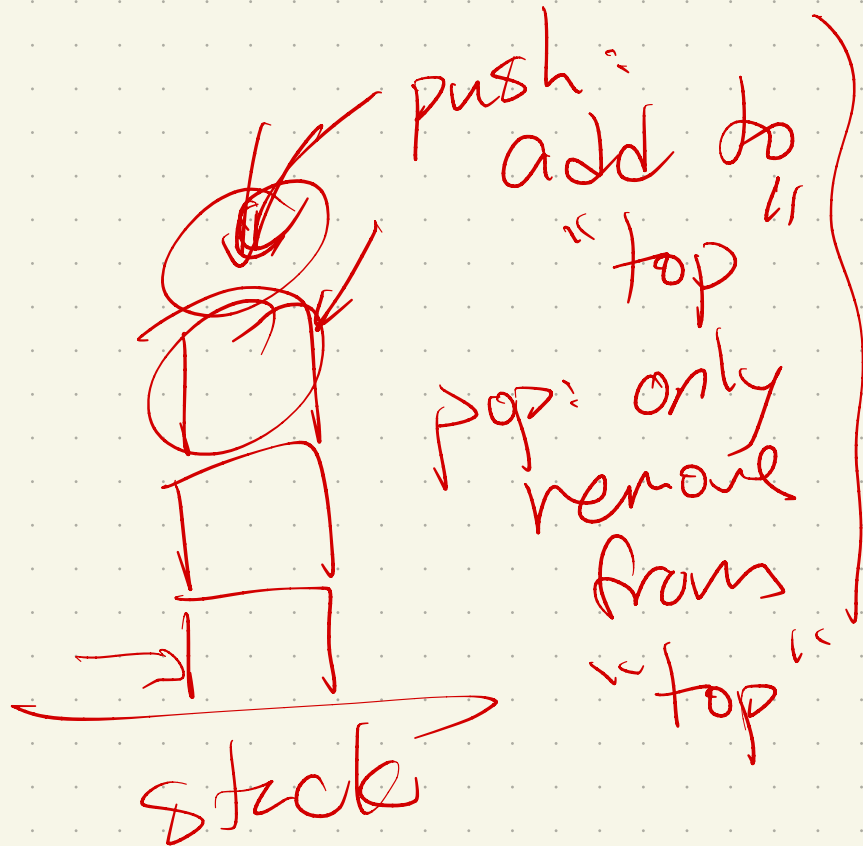


C) either

d) equally bad : $O(n)$

Skipped: Stacks & Queues

push & pop
↗
top



$O(1)$ | push: add to back
pop: remove from front

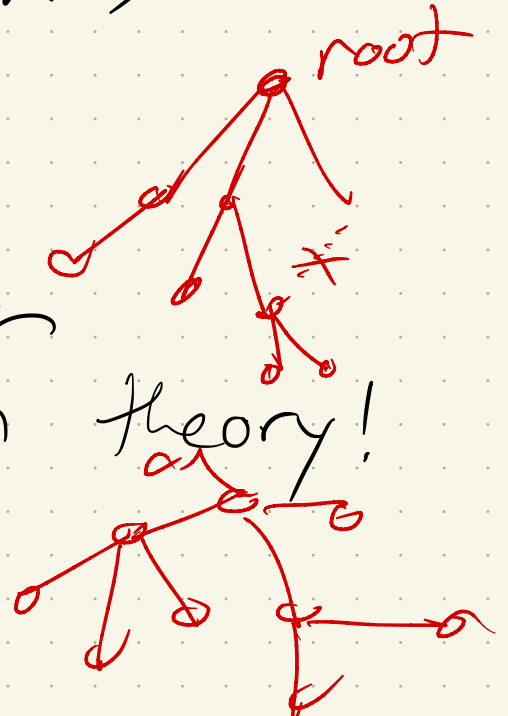
Trees:

Dfn: A tree (in data structures) is a set of nodes which store elements in a parent-child relationship.

Any tree has a special uppermost node
↳ called the root.

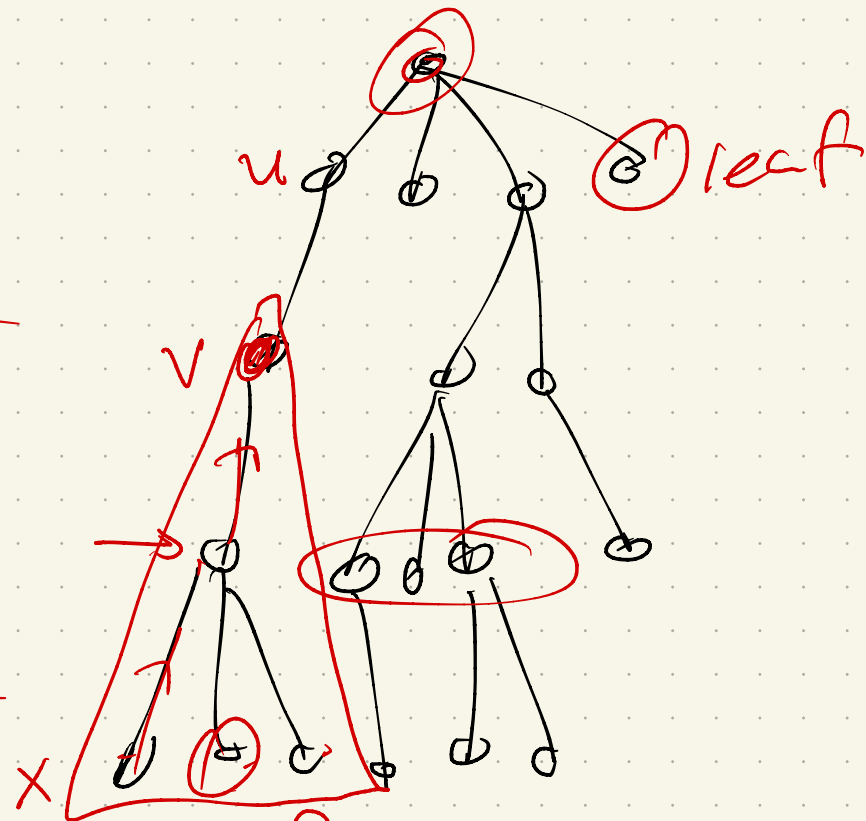
Each node (except the root) has a unique parent.

Note: Not quite the same as in graph theory!
connect + no cycles



More defs

- child / parent
- sibling: common parent
- leaves: no children
- internal nodes: not a leaf
- rooted subtree: specify v ,
↓ take all
descendants
- descendant / ancestor



Practical examples:

Anything w/ parent-child relationship
(rather than linear)

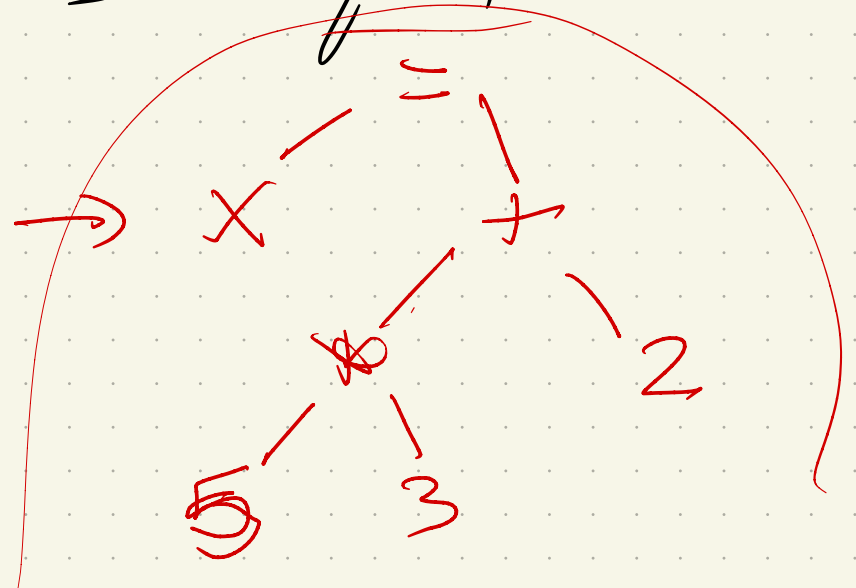
- File systems:

C: Documents / classes / ...

- Family trees: ancestry

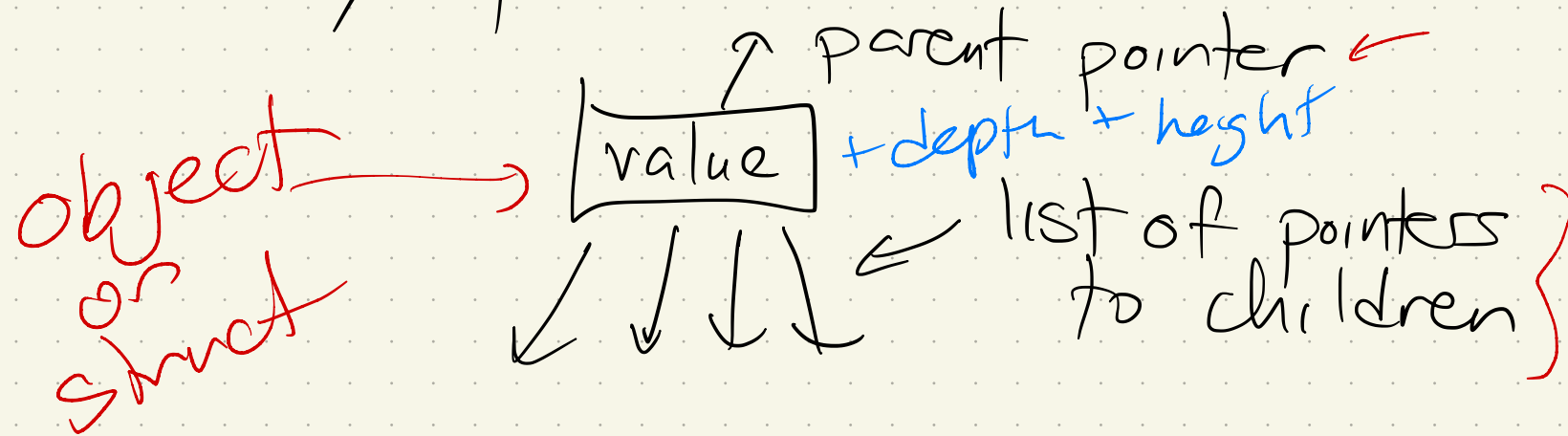
- Parse trees for sentences or equations:

$$x = (5 * 3) + 2$$



A general tree implementation:

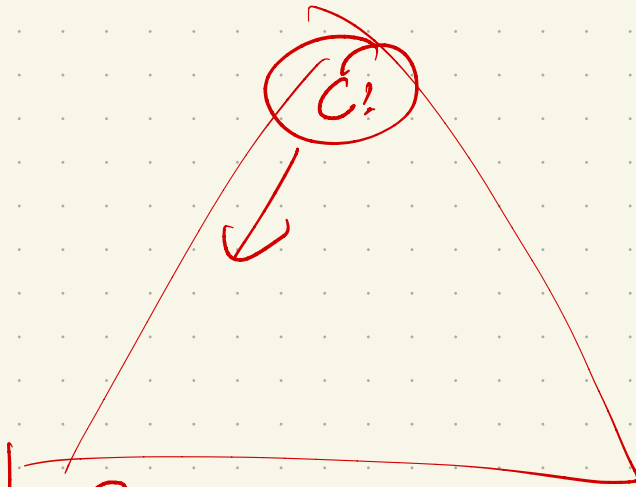
Usually pointer based



Plus global variables:

size: # of nodes

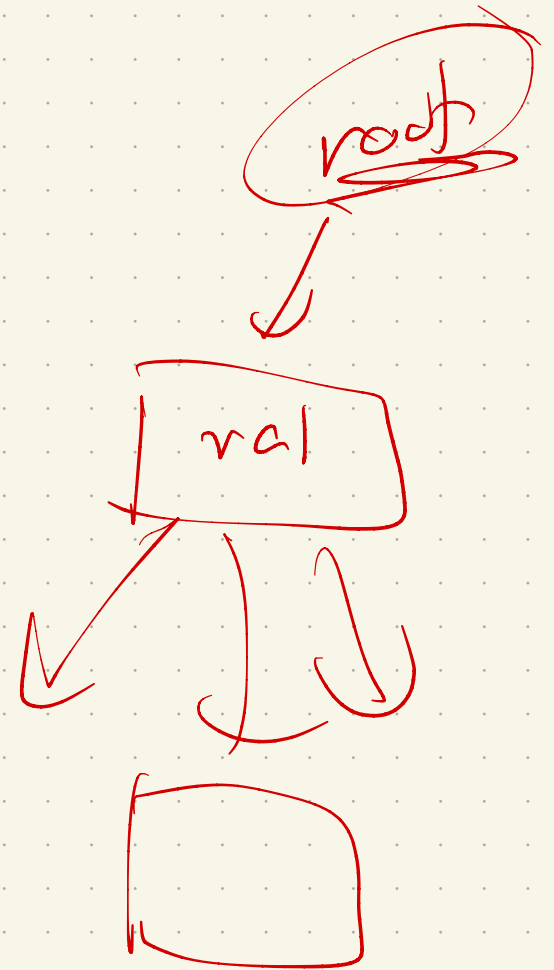
root: entry to tree



tree class {
global root

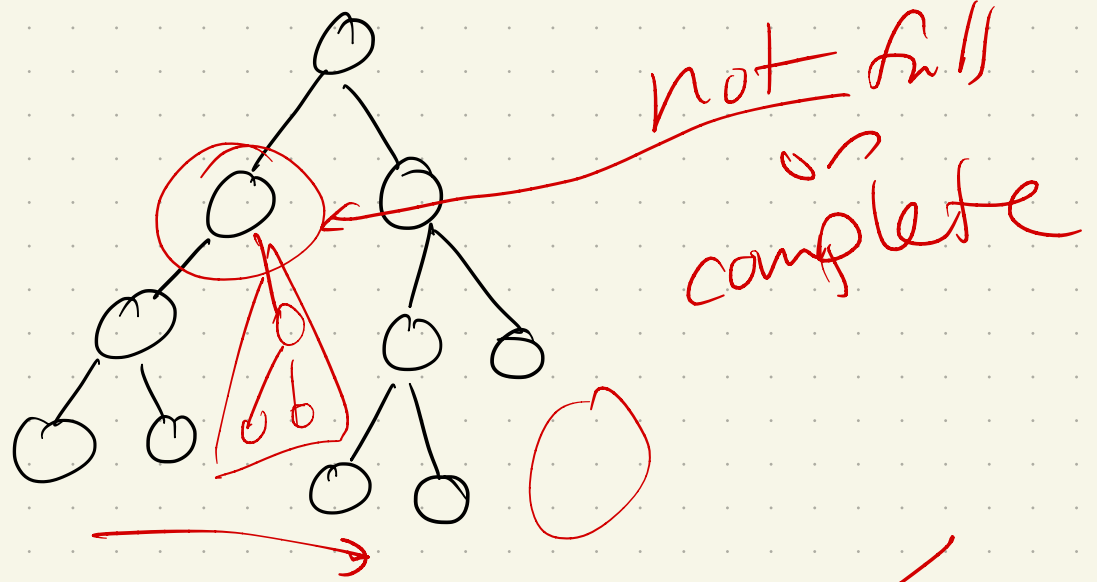
root $\leftarrow \phi$

insert(values)



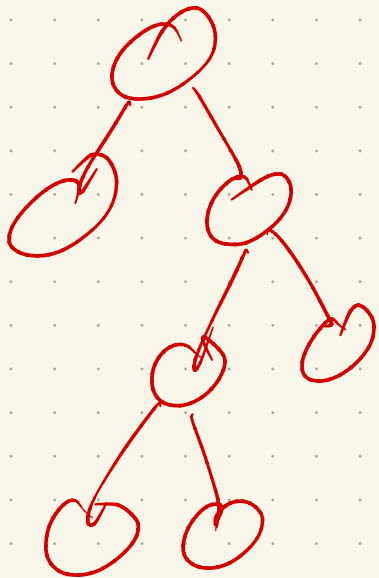
Binary Trees

- Every node has ≤ 2 children:



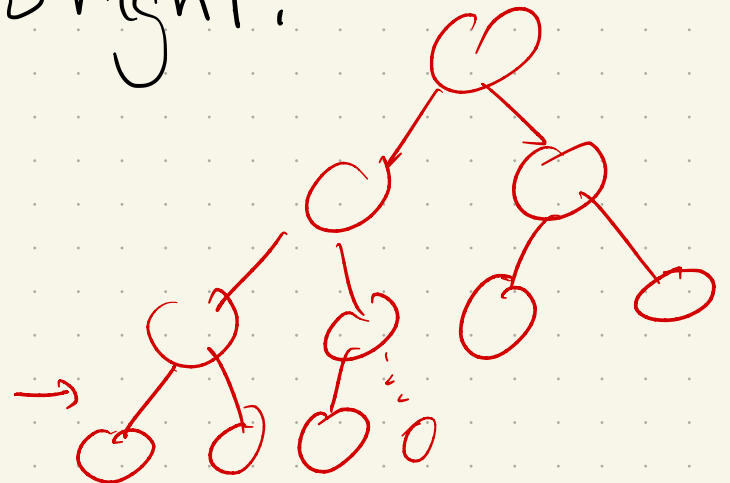
Full tree:

Every node has 0 or 2.



Complete tree:

Every node has 2 except perhaps lowest level which fills left to right.



Depth + Height:

Both defined recursively. 

$$\text{depth}(r) = 0$$

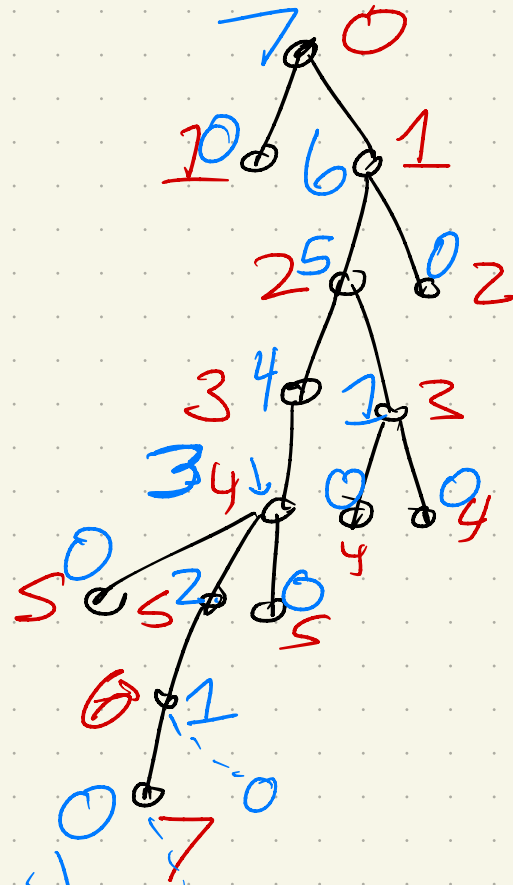
$$\text{depth}(v) = \text{depth}(\text{parent}(v)) + 1$$

$$\text{height}(\text{leaf}) = 0$$

$$\text{height}(v) = \max \{ \text{height of } v\text{'s children} \} + 1$$

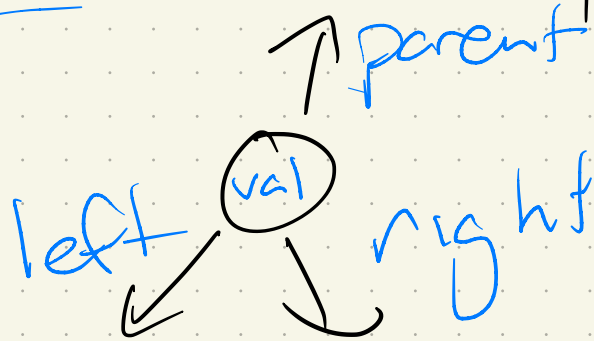
usually = included
↳ often save in node

to compute: $O(n)$ worst case



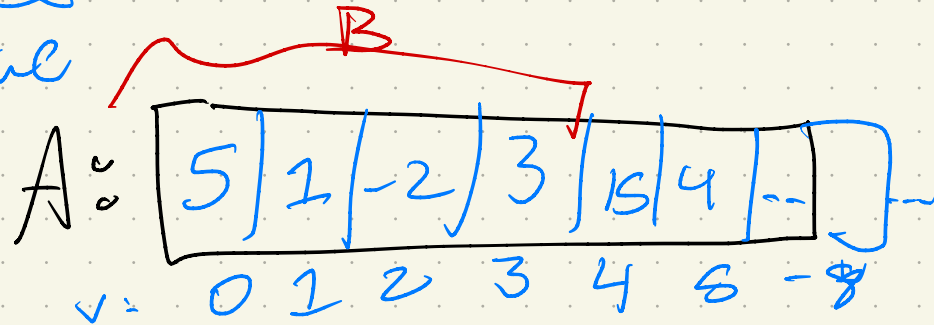
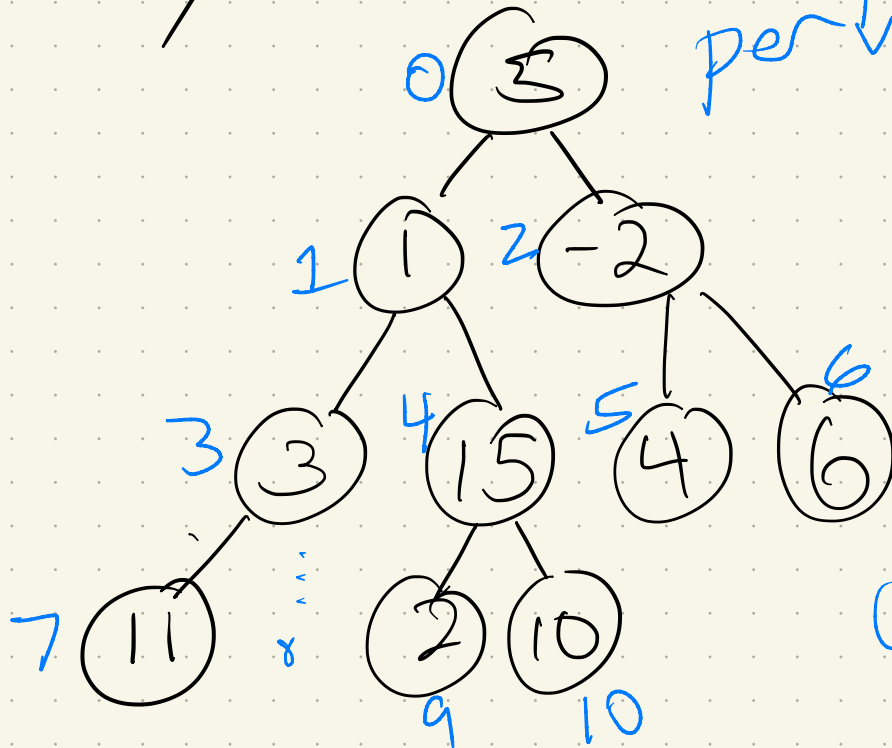
Implementation:

- Pointer based: 3 pointers



```
class Node {  
    value  
    parent  
    left  
    right  
    height  
}
```

- Array based: less space per value



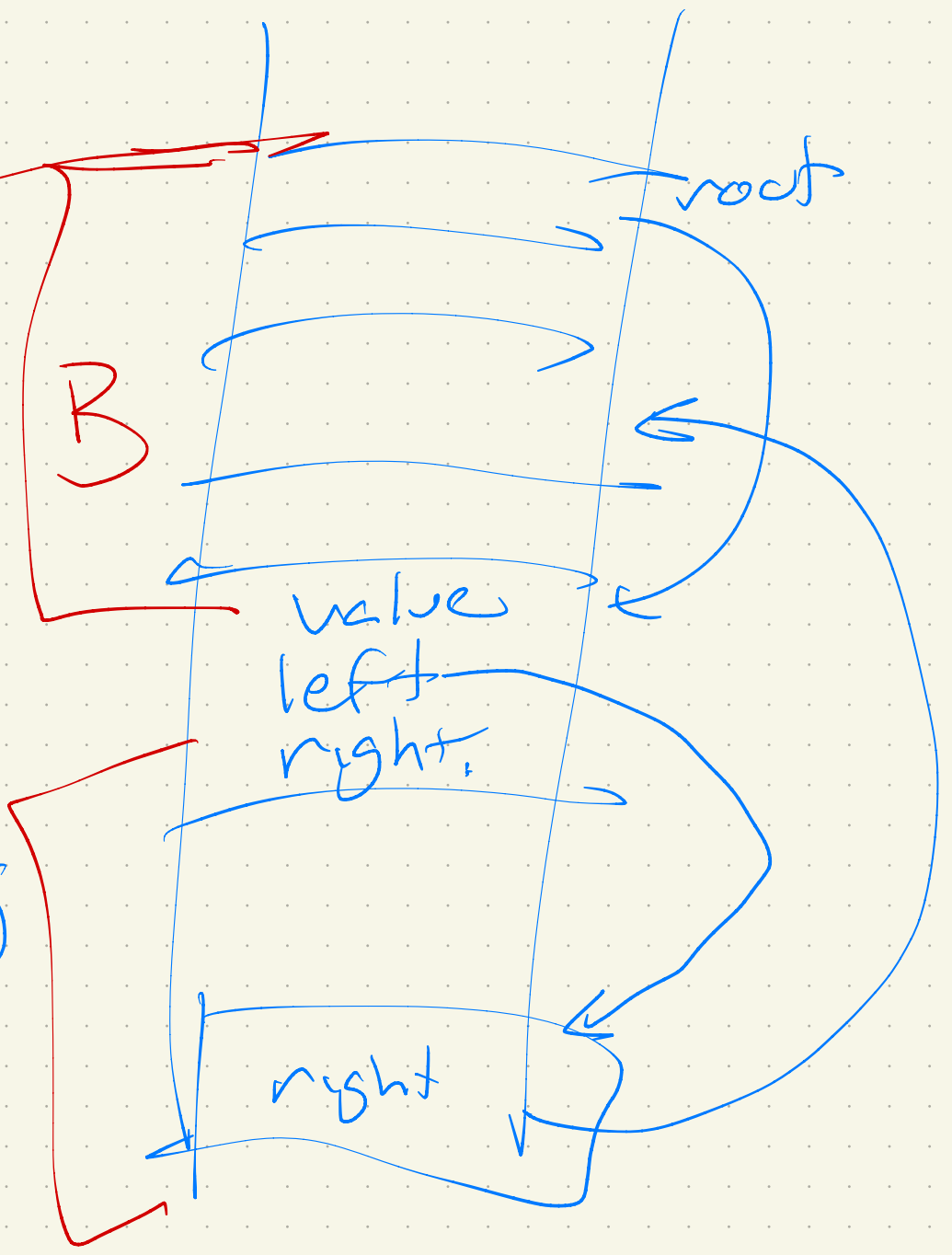
$$\begin{aligned} \text{left}(v) &= 2v + 1 \\ \text{right}(v) &= 2v + 2 \\ \text{parent}(v) &= \left\lfloor \frac{v-1}{2} \right\rfloor \end{aligned}$$

mem

Linked:

traversing
jumps
in
memory
processor

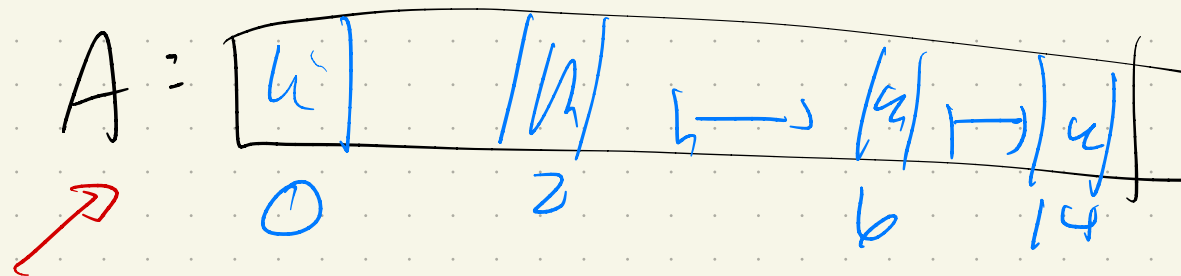
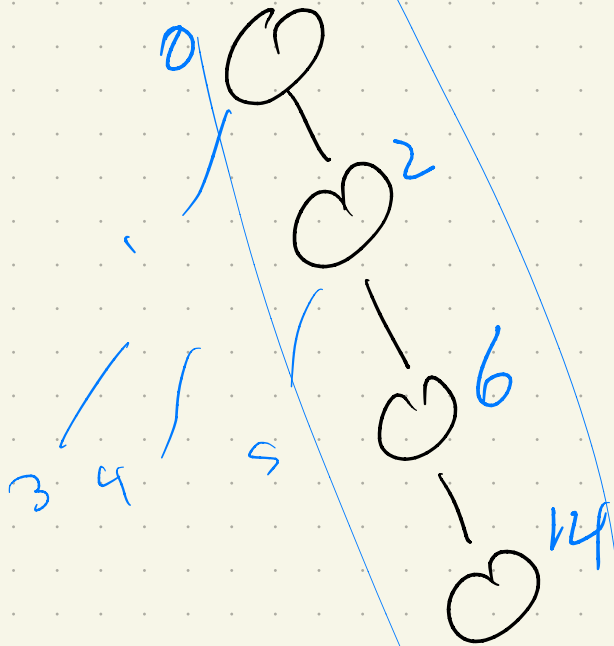
keep looking
at far
parts



Potential downside of array:

Space! sparse tree

Consider:



How big? v nodes

$\hookrightarrow O(2^v)$ space

So use linked tree

Aside: Tree traversals

→ endy point is root

3 ways to traverse a tree:

Starting at root:

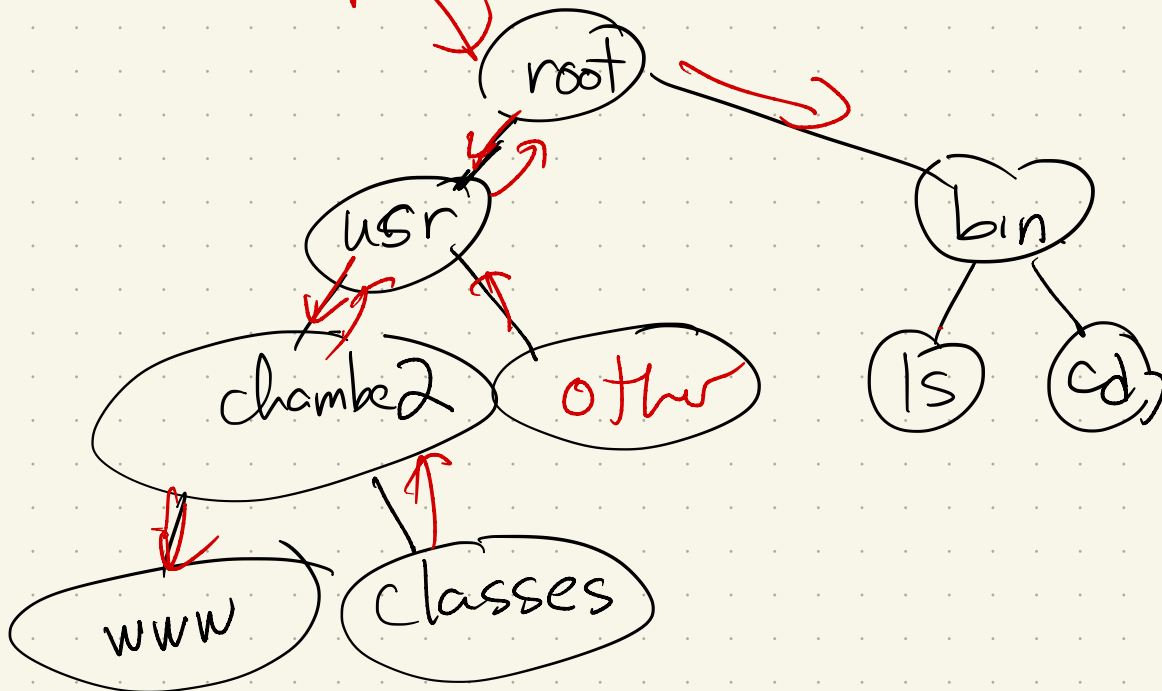
Preorder (v) =

→ visit v (print)

→ Preorder (v → left)

→ Preorder (v → right)

root



preorder (root)

↓

root
↳ usr

↳ chambe2

↳ www

↳ classes

↳ other

↳ bin

↳ ls

↳ cd

;

In order (v) =

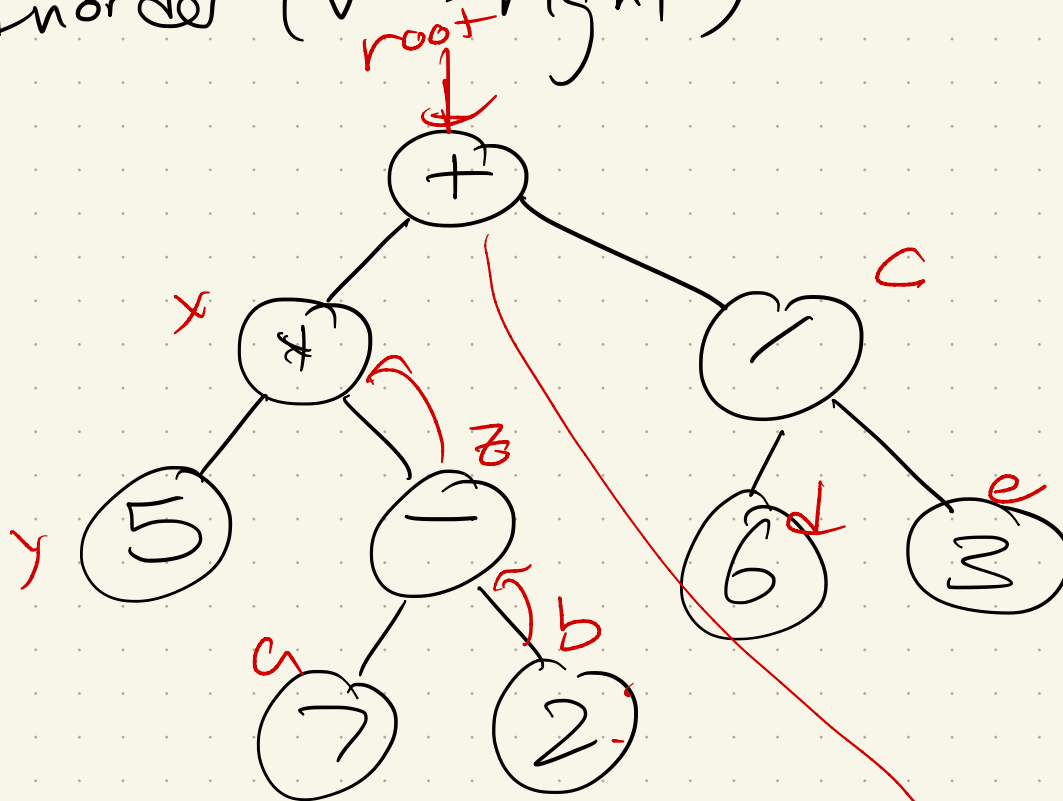
Inorder (v → left)

visit v

Inorder (v → right)

(print)

Ex:



traversal:

$(5 * 7 - 2) + (6 / 3)$
left right

preorder:
 $+(5 - 7 2)$

$(/ 6 3)$

~~v = b~~
~~v = a~~
~~v = z~~
~~v = y~~
~~v = x~~
~~v = root~~
~~v = e~~
~~v = d~~
~~v = c~~
stack

code + operation (input1, input2)

+ (x y)

→ x + y

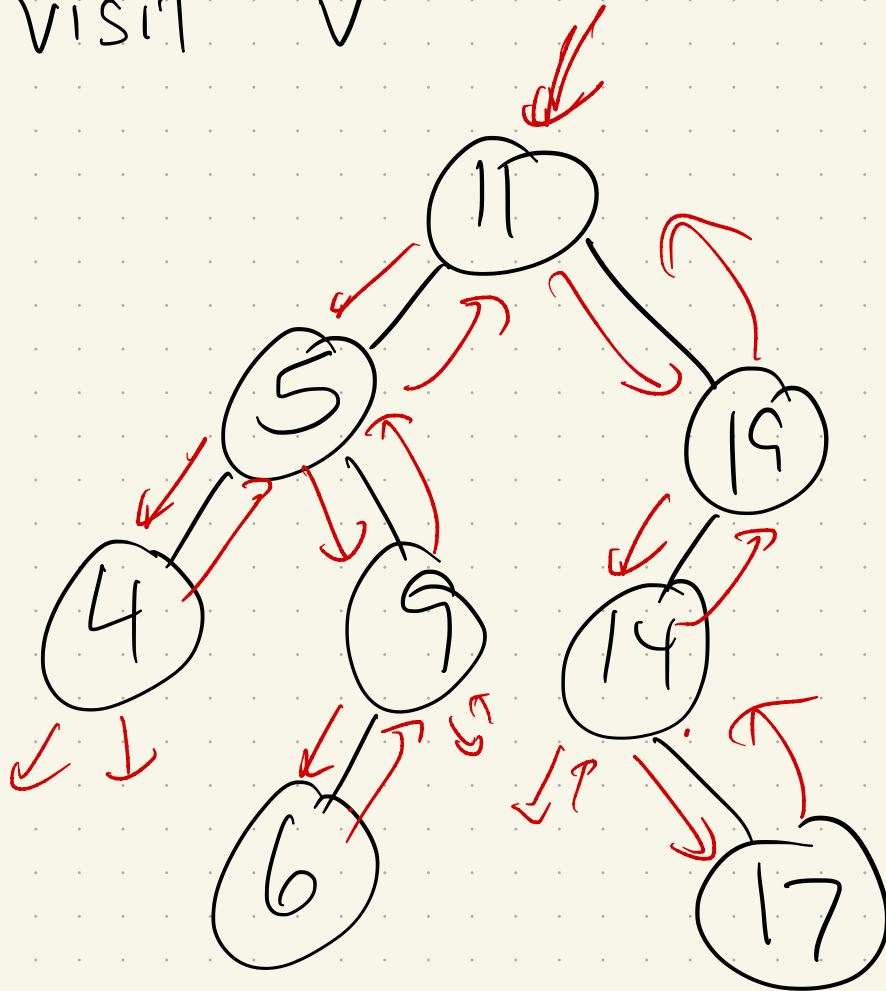
Post order(v) =

post order (v $\rightarrow$ left)
post order (v $\rightarrow$ right)
visit v

Why??

height

Ex:



print:

4, 6, 9, 5,
17, 14, 19,

height(v) =
left $\leftarrow$ height(v $\rightarrow$ left)
right $\leftarrow$ height(v $\rightarrow$ right)
return max(l, r) + 1

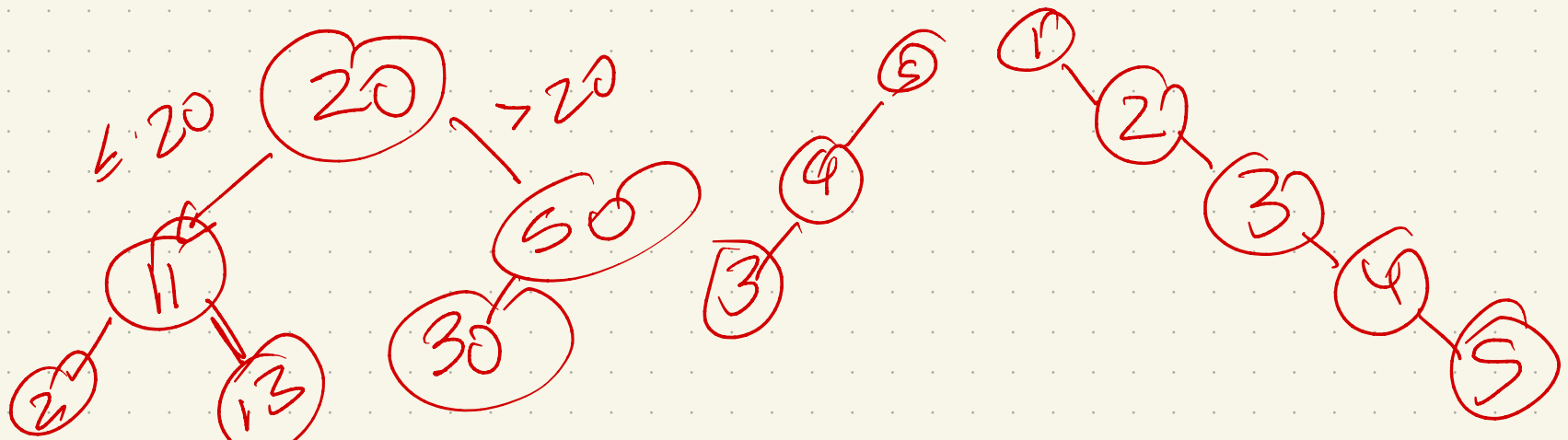
We'll see two tree based data structures:

First: Binary Search Trees

A binary tree where we have a comparison operator on data, & maintain the

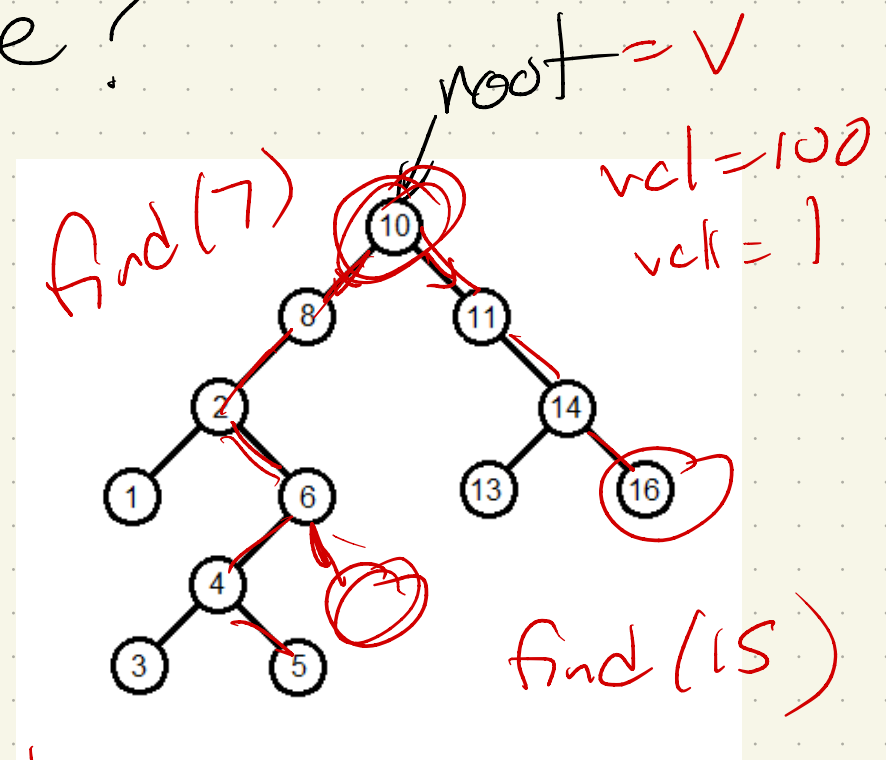
following: • The value at any node is $\geq$ its left child and $<$ its right child.

Ex:



Assuming this data, how can we search for a value?

preorder-ish



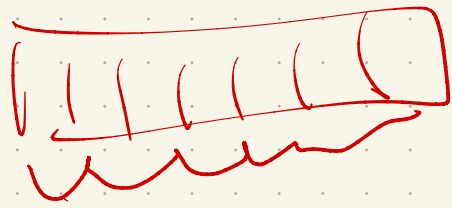
Pseudocode:

base case: $\text{Search}(v, \text{val})$;
if leaf:
return $\text{val} = v \rightarrow \text{data}$ if $v \rightarrow \text{data} = \text{value}$
return true

else
if $\text{val} > v \rightarrow \text{data}$
search($v \rightarrow \text{right}$, val)
else
search($v \rightarrow \text{left}$, val)

Comparison to lists/vectors:

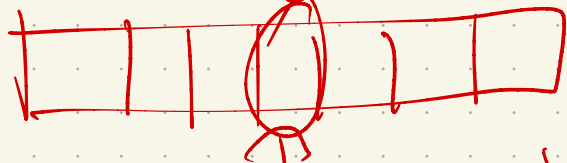
Linear search: if unsorted



$O(n)$ time

Binary search:

sorted vector



check middle

$$T(n) = 1 + T\left(\frac{n}{2}\right)$$

$$\hookrightarrow \log_2 n$$

$$O(\log_2 n)$$

In tree:

traverses a root to leaf path

$$O(\text{depth}(T))$$

Reading:

More on BSTs, & well as
"Balanced" BSTs.

Then: Heaps & Hashing next
week